Functions

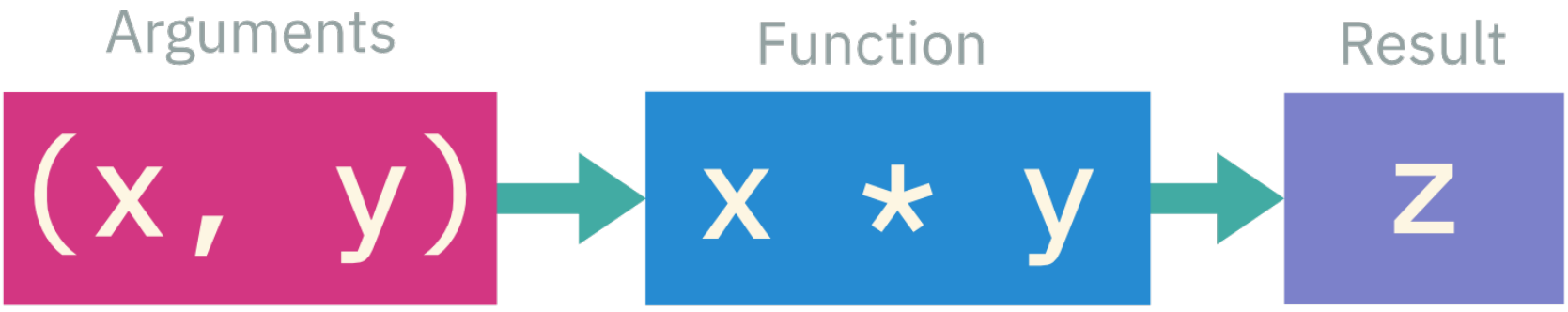
Session overview

1. What is a function?
2. Writing your own functions
3. Things you should know about functions
    - Calling a function
    - Named and positional arguments
    - Default arguments
    - Returning a result
4. Some examples

Functions in R

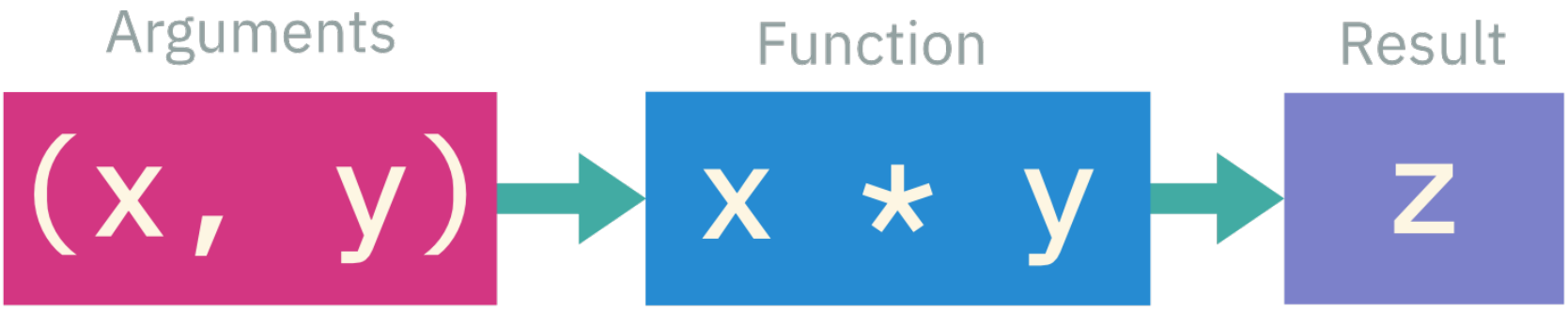
Functions

A function takes inputs, does something with them, and returns the result.



Functions

A function takes inputs, does something with them, and returns the result.



Many functions come with R or are added via packages (e.g., `min`, `mean`, `head`).

But the ability to write your own functions is very powerful.



```r
add_one <- function(x) {
  return(x + 1)
}

add_one(2)
[1] 3
```

```r
multiply <- function(x, y) {
    return(x * y)
}

multiply(2, 3)
[1] 6
```

"A good rule of thumb is to consider writing a function whenever you've copied and pasted a block of code more than twice (i.e. you now have three copies of the same code)."

Hadley Wickham, *R for Data Science (2e)* (2023).

Things you should know about functions

1. Functions are objects.
2. We call a function by typing its name followed by parentheses.
3. Arguments can be named or positional.
4. Arguments can have defaults.
5. We can use `return` within our function to return a result. (But we don't have to).

```
my_exciting_function <- function(x, y) {
    ...
}
```

Things you should know about functions

1. Functions are objects.
2. We call a function by typing its name followed by parentheses.
3. Arguments can be named or positional.
4. Arguments can have defaults.
5. We can use `return` within our function to return a result. (But we don't have to).

```
my_exciting_function()

# What happens if we omit ()?
my_exciting_function
```

Things you should know about functions

1. Functions are objects.
2. We call a function by typing its name followed by parentheses.
3. Arguments can be named or positional.
4. Arguments can have defaults.
5. We can use `return` within our function to return a result. (But we don't have to).

```
# These are equivalent:
rnorm(100, mean = 0, sd = 1)

rnorm(100, 0, 1)
```

Things you should know about functions

1. Functions are objects.
2. We call a function by typing its name followed by parentheses.
3. Arguments can be named or positional.
4. Arguments can have defaults.
5. We can use `return` within our function to return a result. (But we don't have to).

```
my_function <- function(x = 1, y = TRUE) {
    ...
}

rnorm(5)  # Default: mean = 0, sd = 1
```

See help (e.g. `?rnorm`) to learn about default arguments.

Things you should know about functions

1. Functions are objects.
2. We call a function by typing its name followed by parentheses.
3. Arguments can be named or positional.
4. Arguments can have defaults.
5. We can use `return` within our function to return a result. (But we don't have to).

```
my_exciting_function <- function(x, y) {
  return(x * y)
}

my_exciting_function <- function(x, y) {
  x * y
}
```

What's the difference between these three functions?

```
f1 <- function(x, y) {
  return(x * y)
}

f2 <- function(x, y) {
  print(x * y)
}

f3 <- function(x, y) {
  cat(x * y)
}
```

Rolling a die

```
roll_die <- function() {
  sample(1:6, 1)
}

roll_die()
```

Calculating BMI

```r
calculate_bmi <- function(weight, height) {
  weight / (height^2)
}

# Example: 70kg and 1.75m
calculate_bmi(70, 1.75)
```

A function to format the
coefficients from a regression model

```r
p <- function(x) { sprintf("%.1f", x) }

format_coef <- function(est, lo, hi) {
  glue::str_glue(
    "{p(est)} [{p(lo)}, {p(hi)}]"
    )
}

format_coef(1.4188, 1.2103, 1.6120)
# 1.4 [1.2, 1.6]
```

A function to import Qualtrics extracts (`*.xlsx`):

```r
import_qualtrics <- function(path) {
  metadata <- readxl::read_xlsx(path,
    n_max = 2,
    col_names = FALSE,
    .name_repair = "minimal"
  )
  d_name <- metadata[1, ] |>
    as.character() |>
    janitor::make_clean_names()
  d_label <- metadata[2, ] |> as.character()
  data <- readxl::read_xlsx(path, skip = 2,
                            col_names = d_name)
  labelled::var_label(data) <- setNames(d_label,
                                        d_name)
  return(data)
}
```